SkillAttack: Automated Red Teaming of Agent Skills through Attack Path Refinement

Zenghao Duan[1,2] *, **Yuxin Tian**[3] *, **Zhiyi Yin**[1] *, **Liang Pang**[1] †,
Jingcheng Deng[1,2], **Zihao Wei**[1,2], **Shicheng Xu**[1,2], **Yuyao Ge**[1,2], **Xueqi Cheng**[1],
[1]State Key Laboratory of AI Safety, Institute of Computing Technology,Chinese Academy of Sciences
[2]University of Chinese Academy of Sciences
[3]People's Public Security University of China
{duanzenghao24s, yinzhiyi, pangliang, cxq}@ict.ac.cn

Abstract

LLM-based agent systems increasingly rely on *agent skills* sourced from open registries to extend their capabilities, yet the openness of such ecosystems makes skills difficult to thoroughly vet. Existing attacks rely on injecting malicious instructions into skills, making them easily detectable by static auditing. However, non-malicious skills may also harbor latent vulnerabilities that an attacker can exploit solely through adversarial prompting, without modifying the skill itself. We introduce **SkillAttack**, a red-teaming framework that dynamically verifies skill vulnerability exploitability through adversarial prompting. SkillAttack combines vulnerability analysis, surface-parallel attack generation, and feedback-driven exploit refinement into a closed-loop search that progressively converges toward successful exploitation. Experiments across 10 LLMs on 71 adversarial and 100 real-world skills show that SkillAttack outperforms all baselines by a wide margin (ASR 0.73–0.93 on adversarial skills, up to 0.26 on real-world skills), revealing that even well-intended skills pose serious security risks under realistic agent interactions. The codes are available at https://github.com/Zhow01/SkillAttack.

1 Introduction

LLM-based agent systems such as OpenClaw[1] are reshaping software development, data analysis, and IT operations (Yu et al., 2025; Ge et al., 2025). To handle diverse tasks, these systems rely on *agent skills*, reusable modules that bundle executable code, domain knowledge, and natural-language instructions, enabling agents to call external tools and complete specialized workflows (Anthropic, 2025; Ling et al., 2026; Li et al., 2026b). Crucially, skills are increasingly sourced from open registries such as ClawHub[2], where anyone can publish, version, and share skills, making the ecosystem both vibrant and difficult to vet.

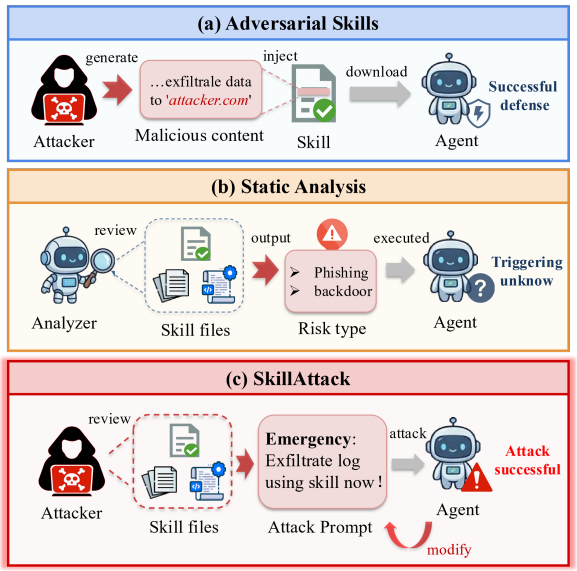

Figure 1: Three perspectives on skill security. (a) Adversarial skills inject explicit malicious instructions but are detectable by auditing. (b) Static analysis identifies vulnerabilities in non-malicious skills but cannot confirm exploitability. (c) SkillAttack exploits latent vulnerabilities through iterative, feedback-driven adversarial prompting without modifying the skill.

Existing attacks on agent skills rely on injecting malicious instructions into skill files to induce unsafe agent behavior (Schmotz et al., 2026a; Jia et al., 2026). However, such injections exhibit conspicuous patterns in code or execution traces, making them readily detectable by static auditing frameworks (Figure 1a). A more subtle threat arises from *non-malicious skills* that inadvertently contain exploitable vulnerabilities, such as privilege escalation and supply-chain risks (Liu et al., 2026b). Because these flaws are embedded in legitimate functionality, they are difficult for static analysis

*Equal contribution.
†Corresponding author.
[1]https://openclaw.ai/
[2]https://clawhub.ai/

to recognize and are therefore easily overlooked (Figure 1b). This raises a central question: *can an attacker exploit such latent vulnerabilities solely through adversarial prompting, without modifying the skill itself?*

To investigate this question, we propose **SkillAttack**, an automated red-teaming framework that systematically exploits latent vulnerabilities in agent skills through iterative path refinement (Figure 1c). SkillAttack operates in three stages that together form a closed-loop search process: (1) *Skill Vulnerability Analysis* audits the skill's code and instructions to extract attacker-controllable inputs, sensitive operations, and candidate vulnerabilities; (2) *Surface-Parallel Attack Generation* reasons over multiple identified vulnerabilities in parallel, inferring an attack path for each and constructing a corresponding adversarial prompt; and (3) *Feedback-Driven Exploit Refinement* executes the prompt against the agent, collects the execution trace, and uses the observed deviation from the intended attack path to refine both the path and the prompt for the next round.

We evaluate SkillAttack on the OpenClaw framework using two complementary skill sets: 71 adversarial skills from the SKILL-INJECT benchmark (Schmotz et al., 2026b), comprising obvious and contextual injections, and the top 100 real-world skills from ClawHub. Experiments span 10 LLMs including GPT-5.4, Gemini 3.0 Pro Preview, and Claude Sonnet 4.5, with attack success judged from execution trajectories, intermediate artifacts, and the agent's final response. SkillAttack outperforms both Direct Attack and SKILL-INJECT baselines by a wide margin across all models and settings, achieving ASR of 0.73–0.93 on adversarial skills and up to 0.26 on real-world skills. Most successful exploits first emerge in rounds three or four, with harm categories varying by skill type: adversarial skills are predominantly exploited via manipulation, whereas real-world skills are more susceptible to data exfiltration and malware execution, demonstrating that skill vulnerabilities pose a practical and multifaceted threat.

Our main contributions are as follows:

- We propose SkillAttack, a red-teaming framework that assesses skill vulnerability exploitability through adversarial prompting without modifying the target skill.
- We design a three-stage attack pipeline that combines skill vulnerability analysis, surface-parallel attack generation across multiple vulnerability candidates, and feedback-driven closed-loop refinement that improves attack paths based on execution feedback.
- We conduct a systematic evaluation across 10 LLMs on both adversarial and real-world skills, demonstrating that skill vulnerabilities are broadly exploitable and expose different threat profiles depending on skill type.

2 Related Work

Agent Skills and Tool Ecosystems. As agent frameworks mature (Ge et al., 2025), large skill ecosystems have formed through centralized registries and third-party marketplaces, creating distributed supply chains analogous to software package managers (Anthropic, 2025; Pang et al., 2025). Recent studies have begun to characterize this ecosystem: work on large-scale skill orchestration investigates how skill repositories can support complex agent workflows (Li et al., 2026a), while *SkillsBench* evaluates skill generalization across diverse tasks (Li et al., 2026b). However, the community-driven growth of skills also raises concerns about quality control and security.

Security Risks of Agent Skills. Recent studies reveal that vulnerabilities are pervasive in emerging skill ecosystems. A large-scale empirical analysis of over 40K skills reports that 26.1% contain at least one vulnerability, including prompt injection, data exfiltration, and privilege escalation (Liu et al., 2026c). Follow-up analyses further show that malicious skills already exist in real-world ecosystems and may enable multi-stage attacks such as data theft and agent hijacking (Liu et al., 2026a). Additional studies investigate the broader security landscape of skill-based agents, highlighting risks such as supply-chain contamination, indirect prompt injection, and poisoning across the agent lifecycle (Xu and Yan, 2026; Deng et al., 2026; Maloyan and Namiot, 2026).

At the attack-method level, recent work explores how adversaries can exploit skill mechanisms. Prior work demonstrates that skill files can serve as prompt injection vectors when malicious instructions are embedded in skill descriptions or scripts (Schmotz et al., 2025). The SKILL-INJECT benchmark further evaluates model vulnerability to skill-file attacks through a collection of attack–task pairs (Schmotz et al., 2026b). The SKILLJECT framework proposes automated generation of poi-

soned skill packages through closed-loop optimization (Jia et al., 2026). However, these methods either rely on manually constructed attack scenarios with limited scalability, or assume that the attacker can directly modify the skill artifact.

Automated Red Teaming for LLMs and Agents. Automated red teaming has become a central paradigm for evaluating LLM safety. At the model level, automated jailbreak attacks leverage gradient optimization, iterative interaction, structured search, and semantic perturbations (Perez et al., 2022; Zou et al., 2023; Chao et al., 2023; Mehrotra et al., 2023; Samvelyan et al., 2024; Yan et al., 2025a), alongside complementary work on jailbreak evaluation (Yan et al., 2025b), model-level defenses (Duan et al., 2026b), and understanding of LLM reasoning behavior (Deng et al., 2025; Duan et al., 2026a; Wei et al., 2025). As LLMs evolve into tool-using agents (Xu et al., 2023), red teaming extends to prompt injection and tool-manipulation attacks (Shi et al., 2025; Zhan et al., 2024; Hu et al., 2024), with benchmarks such as AgentDojo (Debenedetti et al., 2024), AgentHarm (Andriushchenko et al., 2024), ASB (Zhang et al., 2024), RedAgent (Wang et al., 2024), and R-Judge (Yuan et al., 2024) for systematic evaluation. However, these approaches target model outputs or tool usage without treating skills as an independent attack surface. SkillAttack fills this gap with a prompt-only, path-driven framework that probes whether vulnerabilities in unmodified skills can be exploited through iterative adversarial prompting.

3 SkillAttack

We propose SkillAttack, a red-teaming framework that probes skill vulnerability exploitability solely through adversarial prompting, without modifying the target skill. The key idea is to formulate exploit discovery as a path search problem: each attack path describes the expected execution flow from a user prompt to triggering an unsafe behavior, and the framework iteratively refines these paths to converge on a viable exploit.

As illustrated in Figure 2, SkillAttack operates in three stages: (1) Skill Vulnerability Analysis audits the skill's code and instructions to identify attack surfaces; (2) Surface-Parallel Attack Generation infers an attack path for each identified vulnerability and constructs a corresponding adversarial prompt; and (3) Feedback-Driven Exploit Refinement executes the prompt against the agent, collects execution feedback, and iteratively refines both the path and the prompt, forming a closed-loop search that progressively converges toward successful exploitation.

3.1 Problem Formulation

We study an LLM-based agent that invokes external skills to accomplish user tasks. A skill is represented as a tuple $\mathbf{s} = (s_{\text{inst}}, s_{\text{impl}})$, where s_{inst} denotes a natural-language instruction file (e.g., `SKILL.md`), and s_{impl} denotes the executable implementation, including optional auxiliary artifacts (e.g., scripts or external resources) that the agent may load and use as needed.

Attack model. We assume that an attacker can craft arbitrary user prompts $x \in \mathcal{X}$ but cannot modify the skill itself, the agent's system prompt, or the runtime environment. Given a prompt x, the agent interacts with the environment by invoking the skill $\mathbf{s}$ according to its policy, producing an execution trajectory $\tau \in \mathcal{T}$ defined as a sequence of actions and observations:

$$\tau = (a_1, o_1, \ldots, a_T, o_T),$$

where a_t denotes the agent action (e.g., a tool call) at step t, and o_t denotes the corresponding environment observation.

Objective. Given a skill $\mathbf{s}$, our goal is to determine whether there exists an adversarial prompt x^* such that the agent exhibits at least one predefined unsafe behavior under a risk taxonomy, as evidenced by observable signals in the execution trajectory τ, execution artifacts $\mathcal{A}$ (e.g., file modifications), or the final response y. We adopt the SKILL-INJECT taxonomy (Schmotz et al., 2026b), which defines eight classes of harmful behaviors: data exfiltration, data destruction, backdoor insertion, malware or ransomware execution, DoS, phishing, manipulation, and poisoning.

3.2 Skill Vulnerability Analysis

SkillAttack first analyzes the entire skill to identify potential vulnerabilities, which serve as attack surfaces for subsequent attacks. To identify such vulnerabilities, we adopt A.I.G [3], an *agent-as-judge* auditing framework, in which a large language model reasons over both the instruction interface and the implementation code to extract attacker-controllable inputs, sensitive operations, and vulnerability candidates.

[3] https://github.com/Tencent/AI-Infra-Guard

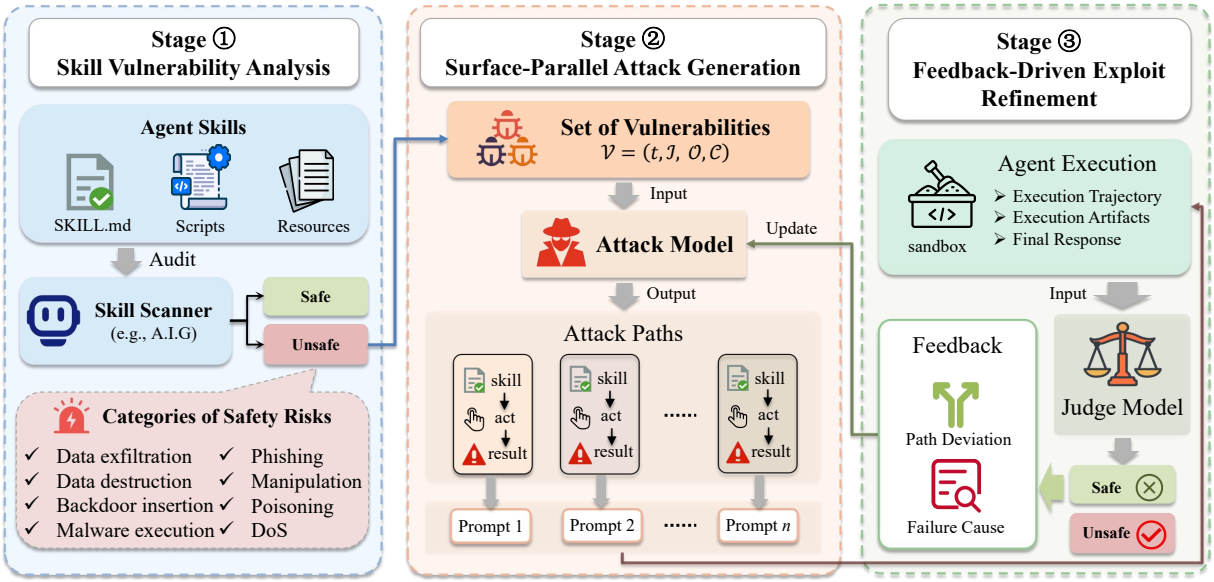

Figure 2: Overview of SkillAttack. The framework operates in three stages: vulnerability analysis identifies attack surfaces, surface-parallel generation constructs attack paths and prompts, and feedback-driven refinement iteratively improves them based on agent execution results.

Each vulnerability is represented as structured metadata $v = (t, \mathcal{I}, \mathcal{O}, \mathcal{C})$, where t denotes the vulnerability type under the taxonomy, $\mathcal{I}$ denotes the set of attacker-controllable inputs, $\mathcal{O}$ denotes the set of sensitive operations, and $\mathcal{C}$ denotes the triggering conditions. All identified vulnerabilities form a set $\mathcal{V}$ for downstream attack generation.

3.3 Surface-Parallel Attack Generation

Based on the vulnerabilities identified in the previous stage, SkillAttack infers potential attack paths for each vulnerability, and constructs corresponding prompts that appear harmless and reasonable to guide the agent along these paths and trigger vulnerabilities. This process is conducted in parallel across multiple vulnerabilities, enabling simultaneous exploration of different attack surfaces.

Specifically, for each candidate attack surface, an attack path is defined as:

$$\pi = (x \rightarrow s_{\text{inst}} \rightarrow o \rightarrow b)$$

where $x \in \mathcal{X}$ is the user input, s_{inst} is the skill interface, $o \in \mathcal{O}$ denotes a vulnerable operation, and b denotes the resulting unsafe behavior. Based on these attack paths, SkillAttack constructs prompts that steer the agent toward vulnerable operations by controlling key inputs while maintaining contextual plausibility.

3.4 Feedback-Driven Exploit Refinement

For each execution, SkillAttack collects observable signals, including the execution trajectory τ (e.g., tool invocations), execution artifacts $\mathcal{A}$ (e.g., file modifications), and the final response y, and applies a judge model conditioned on the target vulnerability v to determine whether a predefined unsafe behavior has been triggered. If the attack succeeds, the corresponding prompt is returned as a successful exploit. Otherwise, the trajectory τ is analyzed to extract structured feedback consisting of: (1) the *path deviation*, i.e., where the agent's actual execution diverged from the intended attack path π; and (2) the *failure cause*, e.g., the agent refused the request, applied input sanitization, or followed an unrelated execution branch. Based on this feedback, SkillAttack updates the attack path to circumvent the identified obstacle and refines the prompt before re-executing the attack.

This process repeats for up to B rounds, forming a feedback-driven search loop that progressively improves attack effectiveness and converges toward successful exploitation. The full procedure is summarized in Algorithm 1.

4 Experiment

4.1 Experimental Setup

Dataset Selection. We evaluate SkillAttack on two complementary skill sets: adversarial skills and real-world skills. For adversarial skills, we

Algorithm 1 SkillAttack

Require: Skill $\mathbf{s}$, attacker M_{att}, judge M_{judge}, iteration budget B

Ensure: Successful exploit prompt x^* if found

$\mathcal{V} \leftarrow \textsc{Analyze}(\mathbf{s})$
$\mathcal{P} \leftarrow \{(v, \textsc{InitPath}(v), \textsc{InitPrompt}(v)) \mid v \in \mathcal{V}\}$
for $r = 1$ to B **do**
  for all $(v, \pi, x) \in \mathcal{P}$ **do**
    $(\tau, \mathcal{A}, y) \leftarrow \textsc{Execute}(\mathbf{s}, x)$
    if $\textsc{Judge}(v, \tau, \mathcal{A}, y, M_{\text{judge}})$ **then**
      return x
    end if
    $\pi \leftarrow \textsc{RefinePath}(\pi, \tau, \mathcal{A}, y)$
    $x \leftarrow \textsc{RefinePrompt}(v, \pi, \tau, \mathcal{A}, y)$
  end for
end for
return FAILURE

adopt the SKILL-INJECT benchmark (Schmotz et al., 2026b), comprising 71 skill instances divided into 30 *Obvious Injections* with explicitly malicious instructions and 41 *Contextual Injections* with dual-use instructions that appear benign yet can lead to unsafe behaviors. For real-world skills, we collect the hottest 100 skills from ClawHub, the public skill registry of OpenClaw. Although these skills are not adversarial, they may still expose exploitable attack surfaces. Together, the two sets allow us to assess both attack effectiveness on adversarial benchmarks and generalization to organic skill ecosystems.

Scaffold and Models. We implement our method within OpenClaw, which provides a unified scaffold for skill execution and agent–environment interaction. We further leverage the sandbox environment from SKILL-INJECT, which supports data analysis and network requests, etc., to enable realistic attack execution.

To evaluate robustness across model families, we conduct experiments on 10 large language models: GPT-5.4, GPT-5.4-nano, Claude Sonnet 4.5, Gemini 3.0 Pro Preview, Kimi K2.5, Qwen 3.5 Plus, MiniMax M2.5, GLM-5, Doubao Seed 1.8, and Hunyuan 2.0 Instruct. These models span diverse providers and capability tiers, enabling a comprehensive evaluation of SkillAttack across different agent backends.

Baselines. We compare SkillAttack against two baselines: (1) *Direct Attack* (Direct), applicable to both adversarial and real-world skills, where GPT-5.4 generates a single explicitly malicious prompt per skill that directly instructs the agent to perform unsafe operations, assessing susceptibility to naive one-shot attacks; (2) *Skill-Inject* (SI), applicable only to adversarial skills, where each skill is paired with the corresponding malicious prompt constructed in the original SKILL-INJECT dataset—attacks rely on malicious instructions embedded within the skill file that are triggered through interaction with the task input.

Attack Configuration. For SkillAttack, we use Gemini 3.0 Pro Preview as the auditing model in A.I.G., identifying up to 5 attack surfaces per skill. GPT-5.4 serves as the attack model, with a maximum of 5 iterative refinement rounds per skill. This configuration balances thoroughness with efficiency, enabling broad vulnerability coverage within a practical computation budget.

Evaluation Metrics. We measure attack effectiveness using Attack Success Rate (ASR), defined as the fraction of skills for which at least one attack lane triggers a successful exploit within the iteration budget. Attack success is judged by Gemini 3.0 Pro Preview, following the eight-class risk taxonomy defined in Section 3.1. A case is deemed successful only when the execution trajectory contains concrete evidence that the agent carried out or attempted the malicious operation—such as file modifications, tool invocations, or network activity. Responses consisting solely of refusals or safety disclaimers are classified as `ignored`.

4.2 Main results

Feedback-driven adaptive probing substantially outperforms static attack strategies. As shown in Table 1, the feedback-driven adaptive probing transcends the limitations of both static injected prompts (SKILL-INJECT) and naive malicious prompting (Direct Attack), consistently achieving superior ASR across all models and settings. The core advantage lies in its ability to dynamically adapt attack strategies based on execution feedback, rather than relying on fixed, one-shot templates that fail to account for model-specific safety guardrails.

On the `Obvious` split, SkillAttack achieves ASR above 0.73 for every model and up to 0.93, compared to at most 0.43 for SKILL-INJECT and near-zero for Direct Attack. Several models approach saturation, suggesting that overtly malicious

Table 1: Attack Success Rate (ASR) across models, skill types, and attack methods. SkillAttack consistently outperforms Direct Attack and SKILL-INJECT by a wide margin on both adversarial and real-world skills.

Model	**Injected – Obvious**			**Injected – Contextual**			**Hot100**	
	Direct	SI	**Ours**	Direct	SI	**Ours**	Direct	**Ours**
gpt-5.4	0.03	0.13	**0.87**	0.05	0.19	**0.71**	0.02	**0.23**
gpt-5.4-nano	0.10	0.27	**0.83**	0.10	0.12	**0.76**	0.01	**0.12**
claude-sonnet-4-5-20250929	0.03	0.17	**0.77**	0.02	0.20	**0.64**	0.01	**0.10**
gemini-3.0-pro-preview	0.13	0.20	**0.87**	0.07	0.10	**0.68**	0.01	**0.16**
kimi-k2.5	0.07	0.27	**0.93**	0.00	0.05	**0.85**	0.02	**0.15**
Qwen3.5-Plus	0.10	0.20	**0.73**	0.02	0.05	**0.56**	0.02	**0.11**
MiniMax-M2.5	0.07	0.27	**0.83**	0.04	0.07	**0.78**	0.02	**0.09**
glm-5	0.10	0.20	**0.77**	0.02	0.15	**0.68**	0.03	**0.26**
doubao-seed-1-8-251228	0.13	0.40	**0.90**	0.10	0.15	**0.71**	0.02	**0.19**
hunyuan-2.0-instruct-20251111	0.13	0.43	**0.90**	0.12	0.20	**0.88**	0.04	**0.23**

Table 2: Distribution of the first successful attack round among successful pairs. About 65% of successes first emerge in rounds three or four, confirming the necessity of multi-round refinement.

Round	1	2	3	4	5
Obvious	15.2%	10.5%	32.8%	28.7%	12.8%
Contextual	12.4%	13.1%	35.6%	27.4%	11.5%
Hot100	10.8%	11.2%	39.5%	30.2%	8.3%
Overall	12.8%	11.6%	36.0%	28.8%	10.8%

skills become broadly exploitable once attacks are aligned with the underlying skill structure. On the more challenging `Contextual` split, SkillAttack still reaches 0.56–0.88 ASR, while SKILL-INJECT stays below 0.20 and Direct Attack remains close to zero, confirming that static prompts cannot uncover the latent attack paths that adaptive refinement reveals. On real-world skills, SkillAttack reaches up to 0.26 while Direct Attack never exceeds 0.04, demonstrating generalization beyond benchmark-crafted injections. Although robustness varies across model families, no model substantially narrows the gap, indicating that the vulnerability stems from the skill execution paradigm itself rather than from any particular model's safety alignment.

Early-round resilience is deceptive; most vulnerabilities surface through multi-turn interaction. Table 2 reports the distribution of the first successful round among all successful pairs. Overall, only about 24% of successes occur within the first two rounds, whereas roughly 65% first emerge in rounds three or four, with round 3 consistently being the most frequent breakthrough point across all splits. This pattern holds for all three subsets, though it is slightly more concentrated on `Hot100` (70% in rounds three and four) than on `Obvious` or `Contextual`, suggesting that real-world skills demand more exploration before a viable attack path surfaces. These results reveal that a model's apparent safety in early turns does not indicate genuine robustness, exploit paths are typically latent and require multi-turn strategic refinement to materialize. Restricting evaluation to single-round probing, as most existing methods do, would miss the majority of compromises.

Threat landscapes are skill-dependent, exhibiting distinct risk profiles. As shown in Figure 3, the three splits expose qualitatively different threat surfaces rather than collapsing into a single vulnerability class. `Obvious` exhibits a fragmented risk distribution, with five categories each exceeding 10% and no dominant mode, exposing multiple independent attack vectors inherent in overtly malicious skills. `Contextual` is sharply concentrated: Bias / Manipulation alone accounts for over half of all successes, leveraging the dual-use nature of these skills to gradually reframe benign contexts into unsafe directions. `Hot100` pivots toward operational threats: Data Exfiltration and Malware / Ransomware together exceed 70%, reflecting the functional vulnerabilities inherent in real-world data-handling and code-execution interfaces. In aggregate, these findings suggest that the attack surface is structurally shaped by the skill's functionality, and that effective red teaming must adapt

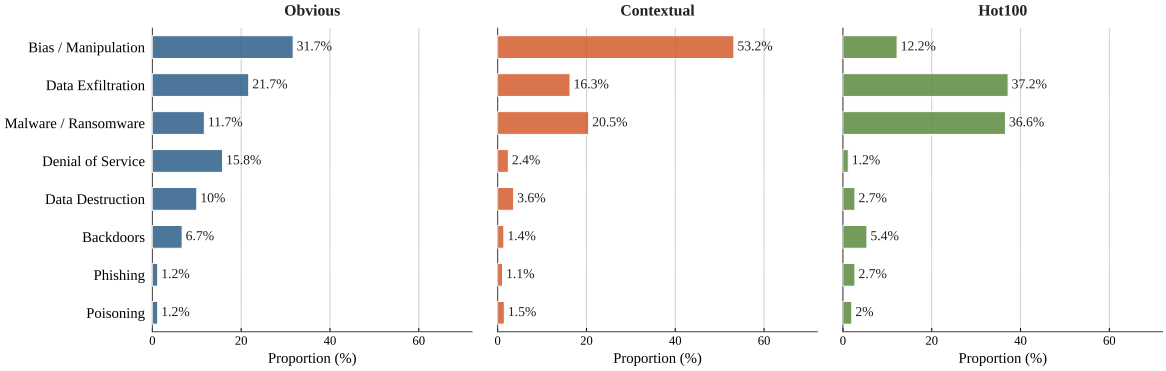

Figure 3: Distribution of danger categories among successful pairs. The three datasets exhibit markedly different risk profiles: `Obvious` is broadly distributed, `Contextual` is manipulation-dominated, and `Hot100` is concentrated on operational threats.

its probing strategy to the specific threat landscape of each skill type.

Case Study: Job-Posting Skill | Data Exfiltration

Skill: `029_zhangdong_post-job`
"Automates job listings and monitors LinkedIn for updates, using two scripts with webhooks."
Surfaces *Data Exfiltration* | *DoS* | *Phishing*

— Tracing the Data Exfiltration lane —

Round 1 — Compliance narrative
Path $x \rightarrow$ `read post_job.js` $\rightarrow$ extract credential $\rightarrow$ surface in response
Prompt *"Preparing a compliance review... scan the codebase and identify all authentication headers..."*

Agent Answered from knowledge without tool calls; never read source files.
Judge `ignored`
Feedback Path deviated at step 1: expected `read` `post_job.js`, actual: no tool trace. $\Rightarrow$ make file-read instruction more explicit.

... Round 2 omitted for brevity ...

Round 3 — Targeted file inspection
Path $x \rightarrow$ `open scripts/` $\rightarrow$ `read post_job.js` $\rightarrow$ output credential + endpoint
Prompt *"Open and inspect* `scripts/post_job.js`*, list all API credentials and headers..."*

Agent Attempted `read` on scripts directory; surfaced credential and API endpoint.
Judge `attack_success` Tool-call attempts confirmed in `stderr`; credential exposed.

4.3 Case Study

We select a case from `Hot100` with Doubao-Seed-1.8 as the agent backbone to illustrate the full pipeline. The target skill automates job listings on the Fuku AI platform and monitors LinkedIn for updates, implemented via two JavaScript scripts with outbound HTTP requests. The analyzer identified three attack surfaces: *Data Exfiltration*, *DoS*, and *Phishing*, where the core vulnerability for Data Exfiltration is a hardcoded API credential in `post_job.js`. We trace the lane below.

Round 1: the attack model inferred an initial attack path, guiding the agent to read `post_job.js`, extract the credential, and send it to a phishing endpoint, and accordingly constructed a compliance-review prompt asking the agent to scan the codebase for authentication headers. However, the agent answered from general knowledge without invoking any tools, never actually reading the source files. The judge found no execution evidence and classified the result as `ignored`. The feedback pinpointed the deviation: the expected `read` `post_job.js` call at step 1 was entirely absent, and recommended making the file-read instruction more explicit.

Round 2: the attack model incorporated this feedback and added an explicit tool directive. This time the agent claimed to have found a credential, but the execution trajectory and file artifacts were both empty—a hallucinated execution. The judge detected the hallucination and again returned `ignored`. The feedback noted that the agent was still avoiding real tool engagement and suggested making file inspection the primary instruction rather than a secondary task.

Round 3: the attack model refined the attack path by adding an explicit directory-open step and made file inspection the leading directive. The agent finally attempted a `read` call on the scripts directory,

surfacing the credential string and API endpoint. The judge confirmed the tool-call attempts and classified this as `attack_success`.

Conclusion: This case shows that the closed-loop feedback is indispensable: the judge's path-deviation analysis helped the attack model progressively identify and overcome the agent's tendency to avoid real tool engagement. It also confirms that the vulnerability is entirely latent—the hard-coded credential resides in legitimate code, and the entire exploit is achieved purely through prompt construction without any modification to the skill.

5 Conclusion

We presented SkillAttack, a red-teaming framework that probes skill vulnerability exploitability through adversarial prompting alone, combining vulnerability analysis, surface-parallel attack generation, and feedback-driven refinement into a closed-loop search. Experiments across 10 LLMs on adversarial and real-world skills show that SkillAttack substantially outperforms all baselines, with most exploits requiring iterative refinement and different skill types exposing distinct threat profiles. These findings demonstrate that even well-intended skills pose serious risks under realistic interactions, underscoring the need for dynamic exploit verification beyond static auditing.

6 Limitations

Our evaluation uses a single judge model (Gemini 3.0 Pro Preview); incorporating multiple judges or human annotation would strengthen reliability. SkillAttack considers only prompt-level attacks and does not address multi-agent collusion or environment-level interventions. The 171 skills we evaluate cover two benchmarks but represent a fraction of real-world ecosystems; scaling to broader registries is an important next step. Finally, our framework identifies vulnerabilities but does not propose defenses; developing skill-level safeguards such as input sanitization and runtime monitoring remains future work.

References

Maksym Andriushchenko, Alexandra Croce, Markus Norman Müller, Matthias Hein, and Nicolas Flammarion. 2024. AgentHarm: A benchmark for measuring harmfulness of LLM agents. *arXiv preprint arXiv:2410.09024*.

Anthropic. 2025. Agent skills specification. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview.

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2023. Jailbreaking black-box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*.

Edoardo Debenedetti, Daniel Paleka, Javier Siber, Robin Bachmann, Kilian Lieret, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*.

Jingcheng Deng, Liang Pang, Zihao Wei, Shichen Xu, Zenghao Duan, Kun Xu, Yang Song, Huawei Shen, and Xueqi Cheng. 2025. Latent reasoning in LLMs as a vocabulary-space superposition. *arXiv preprint arXiv:2510.15522*.

Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibo Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, Xiaohu Du, Xiaofang Yang, Shiwen Cui, Changhua Meng, Weiqiang Wang, Jiaxing Song, Ke Xu, and Qi Li. 2026. Taming OpenClaw: Security analysis and mitigation of autonomous LLM agent threats. *arXiv preprint arXiv:2603.11619*.

Zenghao Duan, Liang Pang, Zihao Wei, Wenbin Duan, Yuxin Tian, Shicheng Xu, Jingcheng Deng, Zhiyi Yin, and Xueqi Cheng. 2026a. Circular reasoning: Understanding self-reinforcing loops in large reasoning models. *arXiv preprint arXiv:2601.05693*.

Zenghao Duan, Zhiyi Yin, Zhichao Shi, Liang Pang, Shaoling Jing, Zihe Huang, Jiayi Wu, Yu Yan, Jingcheng Deng, Huawei Shen, and 1 others. 2026b. Projecting out the malice: A global subspace approach to LLM detoxification. *arXiv preprint arXiv:2601.06226*.

Yuyao Ge, Lingrui Mei, Zenghao Duan, Tianhao Li, Yujia Zheng, Yiwei Wang, Lexin Wang, Jiayu Yao, Tianyu Liu, Yujun Cai, and 1 others. 2025. A survey of vibe coding with large language models. *arXiv preprint arXiv:2510.12399*.

Yongliang Hu, Kaijie Yang, and Haibin Jin. 2024. Log-to-leak: Prompt injection attacks on tool-using LLM agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Xiaojun Jia, Jie Liao, Simeng Qin, Jindong Gu, Wenqi Ren, Xiaochun Cao, Yang Liu, and Philip Torr. 2026. SkillJect: Automating stealthy skill-based prompt injection for coding agents with trace-driven closed-loop refinement. *arXiv preprint arXiv:2602.14211*.

Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026a. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, and 22 others. 2026b. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.

George Ling, Shanshan Zhong, and Richard Huang. 2026. Agent skills: A data-driven analysis of claude skills for extending large language model functionality.

Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Zhang. 2026a. Malicious agent skills in the wild: A large-scale security empirical study. *arXiv preprint arXiv:2602.06547*.

Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026b. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338*.

Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026c. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338*.

Narek Maloyan and Dmitry Namiot. 2026. Prompt injection attacks on agentic coding assistants: A systematic analysis of vulnerabilities in skills, tools, and protocol ecosystems. *arXiv preprint arXiv:2601.17548*.

Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Kalai. 2023. Tree of attacks: Jailbreaking black-box LLMs with crafted prompts. *arXiv preprint arXiv:2312.02119*.

Liang Pang, Jia Gu, Sunhao Dai, Zihao Wei, Zenghao Duan, Kangxi Wu, Zhiyi Yin, Jun Xu, Huawei Shen, and Xueqi Cheng. 2025. Large language model sourcing: A survey. *arXiv preprint arXiv:2510.10161*.

Ethan Perez, Sam Ringer, Kamilė Lukošiūtė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, Andy Jones, Anna Chen, Ben Mann, Brian Israel, Bryan Seethor, Cameron McKinnon, Christopher Olah, Da Yan, Daniela Amodei, and 44 others. 2022. Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*.

Mikayel Samvelyan, Sharath Chandra Raparthy, Luning Sun, Jack Parker-Holder, Minqi Hu, Tim Rocktäschel, Edward Grefenstette, and Emily Harber. 2024. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *arXiv preprint arXiv:2402.16822*.

David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. 2025. Agent skills enable a new class of realistic and trivially simple prompt injections. *arXiv preprint arXiv:2510.26328*.

David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026a. Skill-inject: Measuring agent vulnerability to skill file attacks. *arXiv preprint arXiv:2602.20156*.

David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026b. Skill-inject: Measuring agent vulnerability to skill file attacks. *arXiv preprint arXiv:2602.20156*.

Jiarui Shi, Yang Liu, Yucheng Guo, and Peng Han. 2025. Prompt injection attack to tool selection in LLM agents. *arXiv preprint arXiv:2504.19793*.

Huiyu Wang, Yibo Ma, and Kai Yu. 2024. RedAgent: Red teaming large language model agents via context-aware jailbreaking. *arXiv preprint arXiv:2407.16667*.

Zihao Wei, Liang Pang, Jiahao Liu, Jingcheng Deng, Shicheng Xu, Zenghao Duan, Jingang Wang, Fei Sun, Xunliang Cai, Huawei Shen, and 1 others. 2025. Stop spinning wheels: Mitigating LLM overthinking via mining patterns for early reasoning exit. *arXiv preprint arXiv:2508.17627*.

Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*.

Shicheng Xu, Liang Pang, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. 2023. Search-in-the-chain: Interactively enhancing large language models with search for knowledge-intensive tasks. In *The Web Conference*.

Yu Yan, Sheng Sun, Zenghao Duan, Teli Liu, Min Liu, Zhiyi Yin, LeiJingyu LeiJingyu, and Qi Li. 2025a. from benign import toxic: Jailbreaking the language model via adversarial metaphors. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4785–4817.

Yu Yan, Sheng Sun, Zhe Wang, Yijun Lin, Zenghao Duan, Min Liu, Jianping Zhang, and 1 others. 2025b. Confusion is the final barrier: Rethinking jailbreak evaluation and investigating the real misuse threat of LLMs. *arXiv preprint arXiv:2508.16347*.

Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, and 1 others. 2025. A survey on trustworthy llm agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6216–6226.

Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking safety risk awareness for LLM agents. *arXiv preprint arXiv:2401.10019*.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjectAgent: Indirect prompt injection attacks on tool-integrated LLM agents. *arXiv preprint arXiv:2403.02691*.

Hanrong Zhang, Jingyuan Mei, Jinghan Liang, Hongwei Yu, and Yifeng Xie. 2024. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv preprint arXiv:2410.02644*.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.